FLUTTER DEVELOPMENT - UBIT Batch 23

Assignment 9

Product Catalog Manager

Provider State Management • HTTP API Integration • CRUD Operations

Topic Provider + HTTP (CRUD)	**Difficulty** Intermediate
Submission GitHub repo link	**Marks** 100 pts (see rubric)

1. Overview & Objectives

In this assignment you will build a fully functional Product Catalog Manager — a mobile app where users can view, add, edit, and delete products. Product data lives on a real REST API (crudcrud.com) and local app state is managed with the Provider package. By the end you will have practical experience connecting Flutter UI to a live backend and managing async state in a scalable way.

Learning Objectives

- Use ChangeNotifier and ChangeNotifierProvider to manage global state
- Distinguish when to use Consumer vs Selector vs context.read() vs context.watch()
- Perform HTTP GET, POST, PUT, DELETE using the http package
- Handle loading, error, and empty states in your UI
- Apply a clean, industry-style folder structure
- Build UI that closely matches a provided Figma design

2. Tools & Setup

2.1 crudcrud.com — Your Personal REST API

crudcrud.com gives every student a free, private API endpoint with no sign-up. It supports the four HTTP verbs you need and resets automatically after 24 hours.

1. Open https://crudcrud.com in your browser.
2. Copy the unique endpoint URL displayed on the page. It looks like:

```
https://crudcrud.com/api/YOUR_UNIQUE_ID
```

3. Append /products to that URL — this becomes your products collection:

```
https://crudcrud.com/api/YOUR_UNIQUE_ID/products
```

⚠️ **Important Notes about crudcrud**

Free tier allows 100 requests per 24-hour window — use them wisely during testing.
Your endpoint resets daily, so save your unique URL in a .env file or a constants file.
Data does NOT persist across resets — this is fine for the assignment.
The API auto-generates a field called '_id' for every document. Use this as your product ID.

2.2 API Response Shape

crudcrud returns JSON. A single product looks like this:

Sample JSON — GET /products/:id

```
{
  "_id": "64f3a1b2c7e8d9001a2b3c4d",
  "name": "Architect Chronograph",
  "price": 349.00
}
```

2.3 Flutter Dependencies

Add these to your pubspec.yaml under dependencies:

pubspec.yaml

```
dependencies:
  flutter:
    sdk: flutter
  provider: ^6.1.2        # State management
  http: ^1.2.1            # HTTP requests
```

Then run: flutter pub get

3. App Screens (Figma Reference)

The Figma file linked on the assignment page shows five screens. You must implement all five. The colour palette, typography, and spacing must closely match the designs.

Screen	Description
Loading State	Shown while the product list is being fetched. Displays an animated indicator and skeleton placeholder cards.
Home — Products	Lists all products in cards. Each card shows name, price, and EDIT / DELETE actions. FAB in the bottom-right opens the Add screen.
Empty State	Shown when the API returns zero products. Has an illustration, message, and an Add Product button.
Add Product	Form with Product Name and Price fields. Save Product button calls POST /products.
Edit Product	Same form pre-filled with existing values. Save Changes button calls PUT /products/:id.

4. Recommended Folder Structure

Use this structure from day one. It separates concerns clearly and mirrors what you will find in professional Flutter codebases.

lib/ — folder structure

```
lib/
├── main.dart                    # App entry point + MultiProvider setup
│
├── core/
│   ├── constants/
│   │   └── api_constants.dart   # Base URL, endpoints
│   └── theme/
│       └── app_theme.dart       # Colors, text styles, button themes
│
├── models/
│   └── product.dart             # Product data class + JSON serialisation
│
├── services/
│   └── product_service.dart     # All HTTP calls (no UI logic here)
│
├── providers/
│   └── product_provider.dart    # ChangeNotifier — state + calls service
│
└── views/
    ├── home/
    │   ├── home_screen.dart
    │   └── widgets/
    │       ├── product_card.dart
    │       ├── loading_state.dart
    │       └── empty_state.dart
    └── product_form/
        └── product_form_screen.dart  # Used for both Add and Edit
```

💡 **Why this structure?**

models/ — Pure Dart classes, zero Flutter imports. Easy to unit test.
services/ — All network code in one place. Swap the API URL without touching UI.
providers/ — Business logic and state. No BuildContext needed here.
views/ — Only UI. Reads from Provider, delegates actions to Provider.
This is the separation-of-concerns pattern used in most real Flutter apps.

5. Implementation Guide

5.1 The Product Model (models/product.dart)

Create a plain Dart class. It must be able to construct itself from JSON (fromJson) and convert itself to JSON (toJson). The _id field from crudcrud is a String, not an int.

models/product.dart

```
class Product {
  final String? id;     // nullable: null when creating, assigned by API
```

```
  final String name;
  final double price;

  Product({this.id, required this.name, required this.price});

  // Build a Product from the JSON map crudcrud returns
  factory Product.fromJson(Map<String, dynamic> json) {
    return Product(
      id:    json['_id'] as String?,
      name:  json['name'] as String,
      price: (json['price'] as num).toDouble(),
    );
  }

  // Convert to JSON for POST / PUT requests
  // Do NOT include _id — crudcrud manages that field
  Map<String, dynamic> toJson() => {
    'name': name,
    'price': price,
  };
}
```

5.2 API Constants (core/constants/api_constants.dart)

Never hardcode a URL inside a widget or service method. Keep it in one place:

core/constants/api_constants.dart

```
class ApiConstants {
  // Replace YOUR_UNIQUE_ID with the token from crudcrud.com
  static const String baseUrl =
    'https://crudcrud.com/api/YOUR_UNIQUE_ID';

  static const String productsEndpoint = '$baseUrl/products';
}
```

5.3 Product Service (services/product_service.dart)

The service class contains ONLY network code — no state, no BuildContext. It returns Dart objects (not raw JSON strings) and throws exceptions when something goes wrong.

services/product_service.dart — YOUR TASK: fill in each method body

```
import 'dart:convert';
import 'package:http/http.dart' as http;
import '../core/constants/api_constants.dart';
import '../models/product.dart';

class ProductService {
  final String _endpoint = ApiConstants.productsEndpoint;

  // TODO: Send a GET request to _endpoint.
  // Hint: check statusCode == 200, decode body as a List,
  //       map each item through Product.fromJson().
  // On failure: throw an Exception with the status code.
  Future<List<Product>> fetchProducts() async {
    // your code here
```

```
  }

  // TODO: Send a POST request to _endpoint.
  // Hint: set header 'Content-Type: application/json',
  //       body = jsonEncode(product.toJson()).
  // crudcrud returns 201 on success + the new object in the body.
  Future<Product> createProduct(Product product) async {
    // your code here
  }

  // TODO: Send a PUT request to _endpoint + '/' + product.id.
  // Hint: same headers and body structure as createProduct.
  // crudcrud returns 200 with an empty body on success.
  Future<void> updateProduct(Product product) async {
    // your code here
  }

  // TODO: Send a DELETE request to _endpoint + '/' + id.
  // Hint: no body needed. Throw if statusCode != 200.
  Future<void> deleteProduct(String id) async {
    // your code here
  }
}
```

💡 **HTTP Status Codes to Know**

200 OK — successful GET, PUT, DELETE
201 Created — successful POST (crudcrud returns the new object)
400 Bad Request — your JSON body is malformed
404 Not Found — wrong URL or the _id does not exist
503 / timeout — crudcrud daily limit exceeded, or no internet

5.4 Product Provider (providers/product_provider.dart)

This is the heart of your state management. The provider holds the list of products plus flags for loading and error states. It calls the service, then notifies listeners so the UI rebuilds.

providers/product_provider.dart — YOUR TASK: fill in each method body

```
import 'package:flutter/foundation.dart';
import '../models/product.dart';
import '../services/product_service.dart';

class ProductProvider extends ChangeNotifier {
  final ProductService _service = ProductService();

  // These private fields hold your state.
  // Never expose them directly — use the getters below.
  List<Product> _products = [];
  bool          _isLoading = false;
  String?       _errorMessage;

  // Getters — the UI reads these, never the private fields.
  List<Product> get products     => List.unmodifiable(_products);
  bool          get isLoading    => _isLoading;
  String?       get errorMessage => _errorMessage;
  bool          get hasError     => _errorMessage != null;
  bool          get isEmpty      => !_isLoading && _products.isEmpty;
```

```
  // TODO: Implement fetchProducts().
  // Steps: set _isLoading = true + notifyListeners(), clear _errorMessage,
  // call _service.fetchProducts(), store result in _products,
  // catch any exception into _errorMessage,
  // always set _isLoading = false + notifyListeners() at the end.
  Future<void> fetchProducts() async {
    // your code here
  }

  // TODO: Implement addProduct().
  // Steps: call _service.createProduct(), add the returned Product
  // to _products, then call notifyListeners().
  // Catch exceptions into _errorMessage + notifyListeners().
  Future<void> addProduct(Product product) async {
    // your code here
  }

  // TODO: Implement updateProduct().
  // Steps: call _service.updateProduct(), then find the item in
  // _products by id using indexWhere(), replace it, notifyListeners().
  Future<void> updateProduct(Product product) async {
    // your code here
  }

  // TODO: Implement deleteProduct().
  // Steps: call _service.deleteProduct(id), then remove the item
  // from _products using removeWhere(), then notifyListeners().
  Future<void> deleteProduct(String id) async {
    // your code here
  }
}
```

5.5 App Entry Point (main.dart)

Register all providers at the top of the widget tree using MultiProvider. This makes them available to every screen without passing them down manually.

main.dart

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'providers/product_provider.dart';
import 'views/home/home_screen.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MultiProvider(
      providers: [
        // ChangeNotifierProvider creates AND disposes the provider
        ChangeNotifierProvider(create: (_) => ProductProvider()),
        // Add more providers here as your app grows
      ],
      child: MaterialApp(
```

```
        title: 'Product Catalog',
        debugShowCheckedModeBanner: false,
        theme: ThemeData(/* your theme */),
        home: const HomeScreen(),
      ),
    );
  }
}
```

5.6 Home Screen (views/home/home_screen.dart)

The home screen fetches products once when it first appears, then uses Consumer to react to every state change (loading → data → empty → error).

views/home/home_screen.dart — YOUR TASK: fill in the TODOs

```
class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});
  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {

  @override
  void initState() {
    super.initState();
    // TODO: Call fetchProducts() on the provider here.
    // Remember: use context.read() not context.watch() in initState.
    // Wrap the call in Future.microtask(() => ...) so the widget
    // is fully mounted before you access context.
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Products')),
      // TODO: Wrap the body in a Consumer<ProductProvider>.
      // Inside the builder, check these states in order:
      //   1. provider.isLoading  → return LoadingState()
      //   2. provider.hasError   → return an error widget
      //   3. provider.isEmpty    → return EmptyState()
      //   4. otherwise           → return a ListView of ProductCards
      body: // your code here
      floatingActionButton: FloatingActionButton.extended(
        // TODO: Navigate to ProductFormScreen (no product argument = Add mode)
        onPressed: () { /* your code here */ },
        label: const Text('ADD PRODUCT'),
        icon: const Icon(Icons.add),
      ),
    );
  }
}
```

💡 **initState + Future.microtask — Why?**

You cannot call context.read() synchronously in initState because the widget is still being inserted into the tree and Provider's InheritedWidget hasn't been attached yet.
Future.microtask() schedules the call for the very next event-loop tick, when the widget IS in the tree.

Alternative: use WidgetsBinding.instance.addPostFrameCallback((_) { ... }) for the same effect.

5.7 Provider API — When to Use What

API	Use When…	Example
`context.read<T>()`	Calling a method, NOT reading state. One-time access.	`context.read<ProductProvider>().fetchProducts()`
`context.watch<T>()`	Reading state inside build(). Rebuilds widget on change.	`final p = context.watch<ProductProvider>();`
`Consumer<T>`	You want a sub-tree rebuild without rebuilding the parent.	`Consumer<ProductProvider>(builder: (ctx, p, _) {...})`
`Selector<T, S>`	You only care about ONE field; avoid rebuilds for other fields.	`Selector<ProductProvider, bool>(selector: (_, p) => p.isLoading, ...)`

5.8 Product Form Screen (Shared for Add & Edit)

One screen handles both adding and editing. If a Product object is passed as an argument, you are in edit mode; otherwise you are adding a new product.

views/product_form/product_form_screen.dart — YOUR TASK: fill in the TODOs

```
class ProductFormScreen extends StatefulWidget {
  // If product is null → Add mode.  If not null → Edit mode.
  final Product? product;
  const ProductFormScreen({super.key, this.product});

  @override
  State<ProductFormScreen> createState() => _ProductFormScreenState();
}

class _ProductFormScreenState extends State<ProductFormScreen> {
  final _formKey = GlobalKey<FormState>();
  late final TextEditingController _nameCtrl;
  late final TextEditingController _priceCtrl;
  bool _isSaving = false;

  // Tip: this getter tells you which mode you are in.
  bool get _isEditing => widget.product != null;

  @override
  void initState() {
    super.initState();
    // TODO: Initialize both controllers.
    // In edit mode, pre-fill them with widget.product values.
    // In add mode, leave them empty (or use '' as default).
    // Hint: widget.product?.name ?? ''
  }

  @override
  void dispose() {
    // TODO: Dispose both controllers to free memory.
    super.dispose();
  }
```

```
  Future<void> _save() async {
    // TODO:
    // 1. Validate the form — return early if invalid.
    // 2. Set _isSaving = true (disables the button while saving).
    // 3. Build a Product from the controller values.
    //    In edit mode, carry over widget.product!.id.
    // 4. Call the correct provider method (add or update).
    // 5. After awaiting, check 'if (mounted)' then Navigator.pop().
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(_isEditing ? 'Edit Product' : 'Add Product'),
      ),
      body: Form(
        key: _formKey,
        child: Column(children: [
          // TODO: Add a TextFormField for Product Name.
          // Validator: field must not be empty.

          // TODO: Add a TextFormField for Price.
          // keyboardType: TextInputType.numberWithOptions(decimal: true)
          // Validator: not empty AND double.tryParse() must not return null.

          // TODO: Add a save button.
          // Disable it when _isSaving is true.
          // Label: 'Save Changes' in edit mode, 'Save Product' in add mode.
        ]),
      ),
    );
  }
}
```

6. Submission Checklist

Complete all tasks below. Tick them off before submitting.

#	Task	Details
1	**Folder structure**	Matches the layout in Section 4 exactly.
2	**Product model**	fromJson and toJson implemented and tested manually.
3	**API constants**	Single source of truth for the crudcrud URL.
4	**ProductService**	All four HTTP methods (GET, POST, PUT, DELETE) working.
5	**ProductProvider**	Extends ChangeNotifier, exposes isLoading, products, errorMessage.
6	**MultiProvider in main.dart**	ChangeNotifierProvider registered correctly.
7	**Home screen — Loading state**	Shows while fetching, using isLoading flag from provider.
8	**Home screen — Empty state**	Shown when product list is empty.
9	**Home screen — Product list**	Products rendered from provider; list refreshes on any CRUD action.
10	**Add product form**	Validates fields, calls addProduct(), navigates back on success.
11	**Edit product form**	Pre-fills fields, calls updateProduct(), navigates back on success.

12	**Delete product**	Shows a confirmation dialog, then calls deleteProduct().
13	**Consumer used at least once**	In HomeScreen or ProductCard.
14	**context.read() used correctly**	Only for method calls (not inside build to read state).
15	**Error handling**	Service exceptions caught in provider; error message shown in UI.
16	**UI matches Figma**	Colours, fonts, spacing closely match the provided designs.
17	**Code is clean**	No print statements, no unused imports, consistent indentation.

7. Marking Rubric (100 pts)

Area	Criteria	Pts
Project Setup	Correct pubspec.yaml, folder structure, runs without errors	10
Product Model	fromJson / toJson correct; handles nullable id	10
ProductService	All four HTTP calls implemented and correct	20
ProductProvider	ChangeNotifier setup, all CRUD methods, loading & error flags	20
Home Screen	Loading / empty / error / list states all handled	15
Add & Edit Form	Validation, pre-fill, correct provider method called	15
UI Fidelity	Matches Figma designs (colours, layout, spacing)	5
Code Quality	Clean code, no unused vars, consistent style, no debug prints	5
	TOTAL	**100**

8. Common Mistakes to Avoid

❌ **Mistake 1 — Calling context.watch() inside initState**

Wrong: context.watch<ProductProvider>().fetchProducts(); // in initState
Correct: context.read<ProductProvider>().fetchProducts(); // in initState
Reason: watch() registers a rebuild listener. The widget is not fully mounted in initState, causing a crash.

❌ **Mistake 2 — Calling notifyListeners() too early or forgetting it**

If you update _products but forget notifyListeners(), the UI will never rebuild.
If you call notifyListeners() inside a build(), you get a Flutter assertion error.
Rule: call notifyListeners() only from methods (fetchProducts, addProduct, etc.).

❌ **Mistake 3 — Including _id in your POST body**

toJson() must NOT include the 'id' field. crudcrud assigns _id automatically.
If you send _id in a POST body, crudcrud may reject or duplicate the document.

❌ **Mistake 4 — Hardcoding the crudcrud URL in every file**

Wrong: await http.get(Uri.parse('https://crudcrud.com/api/abc123/products'));
Correct: await http.get(Uri.parse(ApiConstants.productsEndpoint));
Reason: when your 24-hour endpoint changes, you only update ONE file.

❌ **Mistake 5 — Not checking mounted before Navigator.pop()**

After any async operation (await), always check 'if (mounted)' before using context.
If the user navigates away while a request is in flight, the widget is disposed.
Calling Navigator.pop(context) on a disposed widget throws an error.

9. Hints & Bonus Challenges

Helpful Hints

- Use print() liberally during development to inspect API responses — remove them before submitting.
- Test each service method in isolation using a simple main() function or a test file before wiring it to Provider.
- If you get a 503 from crudcrud, you have hit the 100-request limit — wait 24 hours or use a new endpoint.
- Use double.tryParse() for price validation — it returns null instead of throwing an exception.
- The Figma link is in your assignment page — inspect exact hex colours, font sizes, and padding values there.

Bonus Challenges (No Extra Marks — Just Skill)

- Use Selector instead of Consumer for the loading spinner so only the spinner rebuilds, not the whole list.
- Add pull-to-refresh (RefreshIndicator) on the product list.
- Show a SnackBar with a success or error message after each CRUD operation.
- Add a search bar that filters the local list without making a new API call.
- Store the crudcrud URL in a .env file using the flutter_dotenv package.

10. Quick Reference Links

Resource	URL
Figma Design File	https://www.figma.com/design/d0QPqUqXG2AAteAtc3fHam/Assigment-8
crudcrud.com	https://crudcrud.com
Provider pub.dev	https://pub.dev/packages/provider
http pub.dev	https://pub.dev/packages/http

Flutter HTTP cookbook	https://docs.flutter.dev/cookbook/networking/fetch-data
Provider official docs	https://pub.dev/documentation/provider/latest/

📌 Academic Integrity

You may reference official documentation, class notes, and StackOverflow answers.
You may NOT copy a classmate's code or submit code generated entirely by an AI tool.
All code must be written and understood by you. If you use a snippet you found online, add a comment citing the source.
Violations will result in zero marks for the assignment.